Ruido y la era NISQ

Genesis Silesky

20 de octubre de 2026

1. Ruido y la era NISQ

1.1. Objetivo

Ver cómo los errores de puertas hacen aparecer resultados que en un circuito ideal serían imposibles (01 y 10 en un estado de Bell).

1.2. Conceptos previos

Concepto previo: canal de ruido y NISQ

- **Canal de ruido:** una operación que describe cómo un error transforma un estado. El canal *depolarizante* sobre k qubits es $\rho \mapsto (1-\lambda)\rho + \lambda \frac{I}{2^k}$: con probabilidad λ el estado se reemplaza por el estado máximamente mezclado.
- **NISQ** (*Noisy Intermediate-Scale Quantum*, Preskill 2018): dispositivos con decenas a pocos cientos de qubits, con errores, sin corrección de errores a escala.
- **Corrección de errores:** codifica un qubit *lógico* en muchos qubits *físicos*; el costo (cientos o más por qubit lógico, según el código y la tasa de error) es una barrera actual.

1.3. Fundamento teórico

El modelo del código aplica error depolarizante de 1 % tras cada H y de 5 % tras cada CNOT. Las puertas de dos qubits tienen más error que las de uno, lo cual es cualitativamente cierto en hardware real. **Estos valores son ilustrativos y no corresponden a ningún dispositivo.**

Con ruido, el estado de salida deja de ser el estado puro de Bell y las mediciones producen también 01 y 10 con probabilidad pequeña. En la corrida mostrada aparecieron 42 de 2000 shots (2.1 %) con paridad incorrecta.

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 08 - Era NISQ: el ruido produce los resultados 01 y 10

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
# NoiseModel agrupa los errores; depolarizing_error(p, k) es un error
# que, con probabilidad p, reemplaza el estado de k qubits por uno aleatorio.
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit.visualization import plot_histogram

```

```python
# %% Celda 2: circuito y modelo de ruido
# El mismo circuito de Bell de los ejemplos anteriores.
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

ruido = NoiseModel()
# Error de 1% en cada puerta H (puertas de 1 qubit: errores chicos).
ruido.add_all_qubit_quantum_error(depolarizing_error(0.01, 1), ["h"])
# Error de 5% en cada CNOT. En hardware real las puertas de 2 qubits
# son bastante peores que las de 1, por eso se les da mas error.
ruido.add_all_qubit_quantum_error(depolarizing_error(0.05, 2), ["cx"])
# Los valores son ilustrativos, NO miden ningun equipo concreto.

# %% Celda 3: comparar ideal y ruidoso
ideal = AerSimulator().run(qc, shots=2000).result().get_counts()
# Pasar noise_model hace que el simulador inyecte los errores al ejecutar.
noisy = AerSimulator(noise_model=ruido).run(qc, shots=2000).result().get_counts()
print("Ideal  :", ideal)    # solo 00 y 11
print("Ruidoso:", noisy)    # aparecen 01 y 10: errores que rompen la correlacion
# Este es el panorama NISQ (Noisy Intermediate-Scale Quantum): pocos qubits
# y con errores, lo que limita la profundidad de los circuitos utiles.
plot_histogram([ideal, noisy], legend=["ideal", "ruidoso"])
```

1.5. Explicación celda por celda

Celda 2 Mismo circuito de Bell; se crea un `NoiseModel` y se asocian errores a las puertas `h` y `cx`.

Celda 3 Se ejecuta el circuito en un simulador ideal y en uno con ruido (`noise_model=...`) y se comparan los conteos.

1.6. Salida obtenida

```
Ideal  : {'00': 996, '11': 1004}
Ruidoso: {'11': 993, '00': 960, '01': 21, '10': 26}
```

Los números exactos cambian en cada ejecución.

1.7. Interpretación

El ruido rompe la correlación perfecta. En un equipo real también hay errores de lectura, decoherencia, etc., que este modelo simple no incluye. Comparar este histograma con uno obtenido en hardware real (p. ej. IBM Quantum o las SpinQ) es la mejor forma de ilustrar la era NISQ.

1.8. Ejercicios

1. Aumente el error del CNOT a 20 % y observe el cambio.
2. Mida la fidelidad aproximada como la fracción de shots en 00 o 11.
3. Investigue el error de lectura (`ReadoutError`) y agréguelo.